

Reviewer Pack — Mealy Automaticity Lower-Bounds Truth-Table Formula Size

Entry 33ac9cdf1529 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 02:14:10 UTC

Mealy Automaticity Lower-Bounds Truth-Table Formula Size

Entry ID: 33ac9cdf1529 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 02:14:10 UTC

1. Conjecture

Field A (mathematical branch): Finite-state transducer minimization / 2-automaticity of binary sequences (Allouche–Shallit automaticity over the alphabet $\{0,1\}$, distinct from any compression-based complexity) **Field B** (complexity object): Meta-complexity: minimum $\{\text{AND}, \text{OR}, \text{NOT}\}$ -formula leaf count $L(f)$ of a truth table, used as a tractable MCSP proxy

Statement:

For an n -variable Boolean function f with truth table $T_f \in \{0,1\}^{2^n}$, let $A(f)$ be the number of states of the minimum complete DFA over alphabet $\{0,1\}$ that accepts $L_f = \{\text{bin}_n(k) : T_f[k] = 1\}$, where $\text{bin}_n(k)$ is the n -bit MSB-first binary expansion of k . Let $L(f)$ be the leaf-count of the smallest $\{\text{AND}, \text{OR}, \text{NOT}\}$ -formula computing f over the literals $x_1, \dots, x_n, \neg x_1, \dots, \neg x_n$. Conjecture: for every $n \geq 2$ and every f , $A(f) \leq (n+2) \cdot L(f) + 2$; equivalently, $\log_2 A(f) \leq \log_2 L(f) + \log_2(n+2) + o(1)$. A single Boolean function violating this bound refutes the conjecture.

Rationale (proposer’s reasoning):

Mealy/DFA state count is a transducer-theoretic measure that has been intensively studied for infinite automatic sequences but almost never tied directly to MCSP-style formula minimization on truth tables; small formulas should yield few distinguishable prefix-evaluation states, suggesting at most a linear-in- n blow-up from $L(f)$ to $A(f)$. This bridge converts an MCSP question into a poly-time-computable transducer invariant (Hopcroft minimization), giving a falsifiable surrogate without needing to solve MCSP itself. If the bound fails, the offending function exposes a ‘state-rich, formula-cheap’ object — a new natural property candidate that would not relativize, fitting the `META_COMPLEXITY` safe-direction profile.

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): acbe0a5c2b41fcf4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all enumerated f for $n \in \{2,3,4\}$ and ≥ 1000 sampled f for $n \in \{5,6,7,8\}$ (200 random + 50 structured per n), compute $A(f)$ via Hopcroft minimization and $L(f)$ via exact synthesis. SUPPORTED iff every f satisfies $A(f) \leq (n+2) \cdot L(f) + 2$; FALSIFIED iff any single f violates it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - DFA states Boolean function truth table formula size - automaticity Boolean function complexity Allouche Shallit - minimum DFA truth table leaf formula lower bound

Top relevant hits considered: - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1112.2313v2] QBF-Based Boolean Function Bi-Decomposition - [http://arxiv.org/abs/2402.02935v4] Nuclear mass table in deformed relativistic Hartree-Bogoliubov theory in continuum, II: Even- Z nuclei - [http://arxiv.org/abs/2006.04708v2] On three conjectures of P. Barry - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer - [http://arxiv.org/abs/1801.08709v2] Adaptive Lower Bound for Testing Monotonicity on the Line

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import json
from itertools import product

# Helper functions for basic operations
def matrix_multiply(A, B):
    return [[sum(a * b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]

def gaussian_elimination(M):
    n = len(M)
    for i in range(n):
        # Find the pivot
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
```

```

        max_row = j
        M[i], M[max_row] = M[max_row], M[i]

    # Eliminate below
    for j in range(i+1, n):
        factor = M[j][i] / M[i][i]
        for k in range(n):
            M[j][k] -= factor * M[i][k]

    # Back-substitute
    x = [0.0] * n
    for i in range(n-1, -1, -1):
        x[i] = (M[i][-1] - sum(M[i][j] * x[j] for j in range(i+1, n))) / M[i][i]

    return x

def matrix_power(A, k):
    result = [[0]*len(A) for _ in range(len(A))]
    for i in range(len(A)):
        result[i][i] = 1
    while k > 0:
        if k % 2 == 1:
            result = matrix_multiply(result, A)
        A = matrix_multiply(A, A)
        k //= 2
    return result

def hopcroft_minimization(states, transitions, accepting):
    n = len(transitions)
    dfa_states = {frozenset([s]): i for i, s in enumerate(states)}
    dfa_accepting = [i for i, s in enumerate(dfa_states) if any(t in accepting for t in s)]

    def get_next_state(state, char):
        next_states = set()
        for s in state:
            next_states.update(transitions[s][char])
        return frozenset(next_states)

    dfa_transitions = {i: {} for i in range(len(dfa_states))}
    queue = [0]
    visited = [False] * len(dfa_states)
    while queue:
        current_state = queue.pop(0)
        visited[current_state] = True
        for char in '01':
            next_state = get_next_state(dfa_states[current_state], char)
            if next_state not in dfa_states:
                dfa_states[next_state] = len(dfa_states)
            if next_state not in dfa_transitions[current_state]:
                dfa_transitions[current_state][char] = dfa_states[next_state]
            queue.append(dfa_states[next_state])

    return dfa_states, dfa_transitions, dfa_accepting

def truth_table_to_dfa(truth_table, n):
    states = set(range(2**n))

```

```

transitions = {s: {'0': [], '1': []} for s in states}
accepting = []

def get_next_state(state, char):
    next_states = set()
    for i in range(len(state)):
        if state[i] == 1 and truth_table[i][int(char)]:
            next_states.add(i)
    return frozenset(next_states)

for s in states:
    transitions[s]['0'] = get_next_state(s, '0')
    transitions[s]['1'] = get_next_state(s, '1')
    if any(truth_table[i][s] for i in range(len(truth_table))):
        accepting.append(s)

return dfa_states, dfa_transitions, dfa_accepting

def count_leaves(node):
    if isinstance(node, tuple):
        return 1 + sum(count_leaves(child) for child in node)
    else:
        return 0

def generate_truth_table(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n in {5, 8, 11, 14}:
        for _ in range(200):
            truth_table = generate_truth_table(n)
            A_f = hopcroft_minimization(set(range(2**n)), truth_table_to_dfa(truth_table, n)[1], s
            L_f = count_leaves(build_formula(truth_table))
            results.append((A_f, L_f))

    max_ratio = max(A / ((n + 2) * L + 2) for A, L in results)
    conjecture_holds = all(A <= (n + 2) * L + 2 for A, L in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Ratio",
        "metric_value": max_ratio,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(json.dumps({"TRIAL": {"seed": seed, **result}}))
results.append(result)

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
std_ratio = (sum((r["metric_value"] - mean_ratio)**2 for r in results) / len(results))**0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6231a0c0.py", line 139, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6231a0c0.py", line 118, in run_trial
    A_f = hopcroft_minimization(set(range(2**n)), truth_table_to_dfa(truth_table, n)[1], set())[0]
                                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6231a0c0.py", line 95, in truth_table_to_dfa
    transitions[s]['0'] = get_next_state(s, '0')
                           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6231a0c0.py", line 89, in get_next_state
    for i in range(len(state)):
                   ~~~~~^
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `truth_table_to_dfa` before any function was evaluated, so no `A(f)` or `L(f)` data was produced and the pre-registered support condition cannot be checked. | next: Fix the DFA construction (state representation type bug in `get_next_state`) and rerun the enumeration/sampling protocol over $n \in \{2, \dots, 8\}$.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	23512
2	propose	claude_max	opus	0	0	261844
3	preregistration	claude_max	opus	0	0	6857
4	novelty	claude_max	opus	0	0	3071
5	novelty	claude_max	opus	0	0	8237
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24183
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16084
8	judge	claude_max	opus	0	0	3996

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 347786 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/33ac9cdf1529.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/33ac9cdf1529.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/33ac9cdf1529.tar.gz` (if generated)